

2024 年非专业级别软件能力认证第一轮

(CSP-J) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 10 页, 答题纸共有 1 页, 满分 100 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1、BIOS(基本输入输出系统)是一组固化在计算机(C)上一个 ROM 芯片上的程序。

- A. 控制器 B. CPU C. 主板 D. 内存条

2、2KB 的内存能存储(A)个汉字的机内码。

- A. 1024 B. 516 C. 2048 D. 218

3、关于“零”的原码、反码和补码, 下列说法正确的是(C)

- A. 零的原码表示只有一种 B. 零的反码表示只有一种
C. 零的补码表示只有一种 D. 零的原码、反码和补码的表示都有两种

4、一个向量第一个元素的存储地址是 100, 每个元素的长度是 2, 则第 5 个元素的地址是(B)。

- A. 110 B. 108 C. 100 D. 109

5、设 $A=true$, $B=false$, $C=false$, $D=true$, 以下逻辑运算表达式值为真的是(D)

- A. $(A \wedge B) \vee C \wedge D$ B. $((A \wedge B) \vee C) \wedge D$
C. $A \wedge (B \vee C \wedge D)$ D. $(A \wedge B \vee C) \vee D$

6、十进制数 11/128 可用二进制数码序列表示为(D)。

- A. 1011/1000000 B. 1011/100000000 C. 0.001011 D. 0.0001011

7、已知元素(8, 25, 14, 87, 51, 90, 6, 19, 20), 则这些元素以(D)的顺序进入栈, 才能使出栈的顺序满足: 8 在 51 前面; 90 在 87 的后面; 20 在 14 的后面; 25 在 6 的前面; 19 在 90 的后面。

- A. 20, 6, 8, 51, 90, 25, 14, 19, 87 B. 51, 6, 19, 20, 14, 8, 87, 90, 25
C. 19, 20, 90, 8, 6, 25, 51, 14, 87 D. 6, 25, 51, 8, 20, 19, 90, 87, 14

8、设循环队列中数组的下标范围是 $1-n$, 其头尾指针分别为 f 和 r , 则其元素个数为 (D)

- A. $r-f$ B. $r-f+1$ C. $(r-f)\%n+1$ D. $(r-f+n)\%n$

9、用十六进制、八进制和十进制写了如下一个等式: $52-19=33$, 式中三个数是各不相同进位制的数。则 52、19、33 分别为 (B)

- A. 八进制, 十进制, 十六进制 C. 八进制, 十六进制, 十进制
B. 十进制, 十六进制, 八进制 D. 十进制, 八进制, 十六进制

10、在计算机中, ASCII 码是 (A) 位二进制代码。

- A. 8 B. 7 C. 12 D. 16

11、已知小写字母 “m” 的十六进制的 ASCII 码值是 6D, 则小写字母 “c” 的十六进制数的 ASCII 码值是 (D)。

- A. 98 B. 62 C. 99 D. 63

12、用十六位机器码 1110001010000000 来表示定点整数(最高位为符号位), 当它是原码时表示的十进制真值为 -25216; 当它是补码时表示的十进制真值是 (C)

- A. -12608 B. -7551 C. -7552 D. -25216

13. 阅读以下代码片段,, 请问修改给变量 a 赋值 “57 “family”” 正确的语句为 (B)。

```
typedef struct node {
    int data;
    string s;
} *S;
int main() {
    S a;
    return 0;
}
```

- A. $a->data = 57; a->s = \text{"family"};$

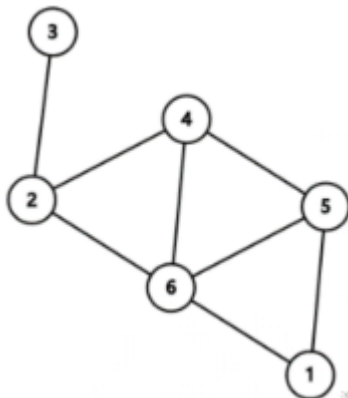
- B. `a = new node; a->data = 57; a->s = "family";`
C. `a.data = 57;a.s = "family";`
D. `a = new node;a.data = 57;a.s = "family";`

14. 已知字符集 {a, b, c, d, e, f, g, h}，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是 (D)。

- A. a c g a b f h B. a d b a g b b
C. a f b e a g d D. a f e e f g d

15. 考虑下图所示的无向图，度最大的结点为 (D) 号结点。

- A. 3
B. 4
C. 5
D. 6



二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
1  #include <iostream>
2
3  void solve1(int n, int k) {
4      k ^= k >> 1;
5      while (~--n)
6          std::cout << (k >> n & 1);
7      std::cout << std::endl;
8  }
9
```

```

10 void solve2(int n, int k, int flag) {
11     if (n == 0) {
12         return;
13     }
14     if (k < (1 << (n - 1))) {
15         std::cout << flag;
16         solve2(n - 1, k, 0);
17     } else {
18         std::cout << (flag ^ 1);
19         solve2(n - 1, k - (1 << (n - 1)), 1);
20     }
21 }
22
23 int main() {
24     int n, k;
25     std::cin >> n >> k;
26     solve1(n, k);
27     solve2(n, k, 0);
28     return 0;
29 }

```

判断题

16. 第二行输出的一定是长度为 n 的若干个 0 或 1 ()
17. 当输入 “5 1” 时，输出的第二行是 “00001” ()
18. 当输入 “10 20” 时，输出的第一行和第二行相等 ()

选择题

19. $\text{solve1}(n, k)$ 的时间复杂度为 ()
- A. n B. $n \log n$ C. n^2 D. 2^n
20. $\text{solve2}(n, k)$ 的时间复杂度为 ()
- A. n B. $n \log n$ C. n^2 D. 2^n
21. 当输入 “6 16”，输出的第一行为 ()
- A. 000000 B. 100100 C. 110110 D. 111111

Solve1 和 solve2 均为求 n 位的第 k 个格雷码

17. ✓ solve1 输出是 $k \gg n \& 1$ ，是 k 先右移 6 位再和 1 做与运算，结果只会是 0 和 1，循环次数是 $\sim n$ ，是循环 n 次（每次会 -1 再判断，判断 $\sim$ 是按位取反，只有全是 1 时会结束循环（即 -1）

18. ✓ 计算即可

19. ✓

20. A

21. A

22. C 可以用 solve2 计算比较方便。

(2)

```
01 #include <iostream>
02 #include <algorithm>
03 #include <vector>
04 using namespace std;
05 const int maxn = 10000;
06 int n;
07 int a[maxn];
08 int method1(int x, int y) {
09     if (x < y) swap(x, y);
10     while (x % y) {
11         x = x % y;
12         swap(x, y);
13     }
14     return y;
15 }
16 int solve1() {
17     if (n == 1) return a[0];
18     int ans = method1(a[0], a[1]);
19     for (int i = 2; i < n; i++) ans = method1(a[i], ans);
20     return ans;
21 }
22 int solve2() {
23     vector<int> b(a, a + n);
24     int l = 0;
```

```

25     while (l < n - 1 && b[n-2] ) {
26         sort(b.begin(), b.end());
27         for (int i = n - 1; i > l; i--) {
28             b[i] -= b[i - 1];
29         }
30         while (b[l] == 0 && l < n - 1) l++;
31     }
32     return b[l];
33 }
34 int main() {
35     cin >> n;
36     for (int i = 0; i < n; i++) cin >> a[i];
37     cout << solve1() << endl;
38     cout << solve2() << endl;
39     return 0;
40 }

```

假设输入的数字都是小于 10000 的正整数，完成下面的判断题和单选题：

判断题

23. 如果将第 30 行代码删去，输出不变 ()
24. 当输入 “5 1 2 3 4 5”，输出的第二行大于第一行 ()
25. 当输入是 “4 99 999 9999 99999”，输出的第一行等于第二行 ()

单选题

26. solve1() 的时间复杂度是 ()
- A. n B. $n \log n$ C. n^2 D. 2^n
27. solve2() 的时间复杂度是 ()
- A. n B. $n \log n$ C. n^2 D. 2^n
28. 当输入是 “6 24 36 48 200 96 112” 时 ()
- A. 4 B. 6 C. 8 D. 11

Solve1 和 solve2 都是求的 n 个数的最大公约数，solve1 是循环每一个数，维护一个 ans 表示前 $i-1$ 个数的最大公约数，然后用 ans 去和第 i 个数求最大公约数（method1 是辗转相除法）后更新 ans。Solve2 是对数组排序，然后用大数减去小数（更相减损术）

23. $\checkmark$ 30 行的作用是过滤掉减去的 0。循环出口是 $l=n-1$ 或者 $b[n-2]==0$ 都可以，如果删除 30 行如果只有 $l<n-1$ 则 while 循环无法退出（有 $b[n-2]>0$ 可以保证退出），如果只有 $b[n-2]>0$ ，当 $n=1$ 时则会导致数组越界

24. $\times$ solve1 和 solve2 永远相同

25. $\checkmark$

26. A 每个数做一次循环，欧几里得算法求最大公约数和数值的位数有关，由于数字最多只有 4 位，可以视为常数级。

27. C 循环是直到减到数组内只有一个数，其余都是 0，这个过程是 n^2 的（另外数组前缀有大量的 0，在用 sort 快速排序的时候效率也是 n^2 的）

28. A 24 36 48 200 96 112 的最大公约数是 4

三、完善程序（单选题，每小题 3 分、合计 30 分）

(1) (枚举因数) 从小到大打印正整数 n 的所有正因数。试补全枚举程序。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 int main() {
05     int n;
06     cin >> n;
07     vector<int> fac;
08     fac.reserve((int)ceil(sqrt(n)));
09     int i;
10     for (int i = 1; i * i < n; ++i) {
11         if (①)
12             fac.push_back(i);
13     }
14     for (int k = 0; k < fac.size(); k++)
15         cout << ② << " ";
16     if (③)
17         cout << ④ << " ";
18     for (int k = fac.size() - 1; k >= 0; k--)
```

```

19         cout << ⑤ << " ";
20     return 0;
21 }

```

29. ①处应填 (A)。

- A. $n \% i == 0$ B. $n \% i == 1$
 C. $n \% (i-1) == 0$ D. $n \% (i-1) == 1$

30. ②处应填 (B)。

- A. $n / \text{fac}[k]$ B. $\text{fac}[k]$
 C. $\text{fac}[k]-1$ D. $n / (\text{fac}[k]-1)$

31. ③处应填 (C)。

- A. $(i-1) * (i-1) == n$ B. $(i-1) * i == n$
 C. $i * i == n$ D. $i * (i-1) == n$

32. ④处应填 (D)。

- A. $n-i$ B. $n-i+1$ C. $i-1$ D. i

33. ⑤处应填 (A)。

- A. $n / \text{fac}[k]$ B. $\text{fac}[k]$
 C. $\text{fac}[k]-1$ D. $n / (\text{fac}[k]-1)$

(2) (求解数独) 输入一个 9×9 的矩阵, 1-9 表示数独题面, 0 表示题面待填的空格, 最终使得 1-9 每个数字在每一行、每一列和每一宫中都只出现一次, 如果题面有解, 则输出任意数独的答案, 如果题面误解, 则输出 “NO”。

在程序中, 按顺序枚举每个空填 1-9, 每填一格就检查新填的这格是否符合规则, 如果不符合则回溯。

试补全程序。

```

01 #include<iostream>
02 #include<vector>
03 using namespace std;
04 const int z = ①;
05 int map[10][10];
06 struct block {
07     int x, y;
08 };
09 vector<block> list;
10

```



```

11 bool check(int x, int y) {
12     for (int i = 0; i < 9; i++)
13         if (map[i][y] == map[x][y] && i != x)
14             return false;
15     for (int i = 0; i < 9; i++)
16         if (map[x][i] == map[x][y] && i != y)
17             return false;
18     for (int i = x / z * z; i < x / z * z + z; i++)
19         for (int j = y / z * z; j < y / z * z + z; j++)
20             if (②)
21                 return false;
22     return true;
23 }
24
25 bool findAns = false;
26 void dfs(int p) {
27     if (p == list.size()) {
28         ③;
29         return;
30     }
31     block n = list[p];
32     for (int i = 1; i <= 9; i++) {
33         map[n.x][n.y] = i;
34         if (④) dfs(p + 1);
35         if (findAns) return;
36     }
37     ⑤;
38 }
39
40 int main() {
41     for (int i = 0; i < 9; i++) {
42         for (int j = 0; j < 9; j++) {
43             cin >> map[i][j];
44             if (map[i][j] == 0) list.push_back({ i, j });
45         }

```

```

46     }
47     dfs(0);
48     if (!findAns) cout << "NO";
49     else {
50         for (int i = 0; i < 9; i++) {
51             for (int j = 0; j < 9; j++) cout << map[i][j] << " ";
52             cout << endl;
53         }
54     }
55     return 0;
56 }

```

1. ①处应填()

- A. 0 B. 3 C. 6 D. 9

2. ②处应填()

- A. `map[i][j] == map[x][y] && (i != x || j != y)`
 B. `map[i][j] == map[x][y] && i != x && j != y`
 C. `map[j][i] == map[x][y] && (i != x || j != y)`
 D. `map[j][i] == map[x][y] && i != x && j != y`

3. ③处应填()

- A. `p=0` B. `findAns = true`
 C. `p--` D. `list.clear()`

4. ④处应填()

- A. `check(n.i, n.j)`
 B. `check(n.x, n.y)`
 C. `check(i, j)`
 D. `check(x, y)`

5. ⑤处应填()

- A. `p++` B. `map[n.x][n.y]=0`
 C. `findAns = true` D. `return false`

DFS 枚举求数独的解。先 (41-45 行) 记录下所有待填的空 (list)，然后按顺序枚举每个空填 1-9，每填一格就检查新填的这格是否符合规则，如果不符合则回溯，如果 1-9 都不符合则再回溯到上一个空格。

1.B 12-14 行是检查列，12-17 行是检查行，明显 18-21 行是检查宫，i 和 j 分别枚举行和列 (3×3 为 1 宫)

2.A 数字如果和(x,y)处相同 (当然 x,y 这个位置除外) 那么不符合规则，需要返回 false

3.B 当所有空都被填上退出 dfs 递归 (可以见 34 行，p+1 表示填下一个空，p 从 0 开始，表示要填的空的编号)

4.B 每填一格就检查新填的这格是否符合规则 (另外几个选项都是语法错误)

5.B 如果 32 行的循环 1-9 都失败，那么(在函数结束后会自动返回上一个待填的空，即上一层函数)，为了不影响之前的空，需要把这个空清空。